

软件过程改进方案设计文档

本方案以智能交通 OD 预测项目为对象，围绕“AI 技术如何改造软件过程”展开设计。文档不把改进理解为单纯增加工具，而是把需求、数据、模型、验证、发布和监控重构为可度量、可审计、可反馈的过程链条。

1 原有过程诊断

原有过程的主要问题不是“没有开发”，而是开发活动缺少稳定的过程边界。需求说明常由聊天记录、临时文档和实验备注拼接而成；数据清洗脚本随实验不断复制，版本差异难以追踪；模型参数写入代码或命令行，复现实验时需要人工回忆；结果报告依赖手动整理，图表、指标和结论之间存在错位风险。

这类问题在课程项目中并不少见，却会直接削弱软件过程质量。一次模型训练失败，表面看是参数问题；向下追问，可能是数据切分日期变了，也可能是天气字段缺失率上升，还可能是同名脚本覆盖了旧版本。没有过程证据链，团队只能凭经验排查。经验当然有价值，可一旦项目进入多轮迭代，经验就会变成不可复制的隐性成本。

问题域	具体表现	影响范围	优先级
需求管理	目标、指标和验收口径分散在多处文本中	影响模型选择与结果解释	高
数据治理	样本切分、缺失处理和网格映射缺乏版本记录	影响训练稳定性与复现实验	高
配置管理	代码、参数、数据和结果未绑定	影响对比实验可信度	高
验证评估	过度依赖总体 MAE，缺少高流量 OD 分析	影响业务价值判断	中高
过程监控	上线后缺少数据漂移和性能退化观察	影响持续运行安全	中高

表 1 原有软件过程问题诊断

2 改进目标与原则

改进目标被设定为四个可量化方向：实验复现时间从 40 分钟压缩到 15 分钟以内；关键实验记录完整率提升到 95% 以上；数据质量缺陷发现率提升到 80% 以上；模型异常告

警定位时间控制在 2 小时以内。目标之所以写成数字，是为了避免“加强管理”“提高质量”这类空泛表述。过程改进必须接受度量，否则无法判断改进是否真实发生。

改进原则则更克制：不追求一次性建设庞大平台，而是用最小闭环覆盖项目风险最高的环节。对课程项目而言，需求模板、数据版本表、实验追踪表、自动化评估脚本和结果审查清单，已经足以构成一个可运行的过程骨架。AI 技术嵌入其中，承担辅助分析、文档生成、日志归纳和异常提示，而不是替代过程负责人。

3 改进后过程框架

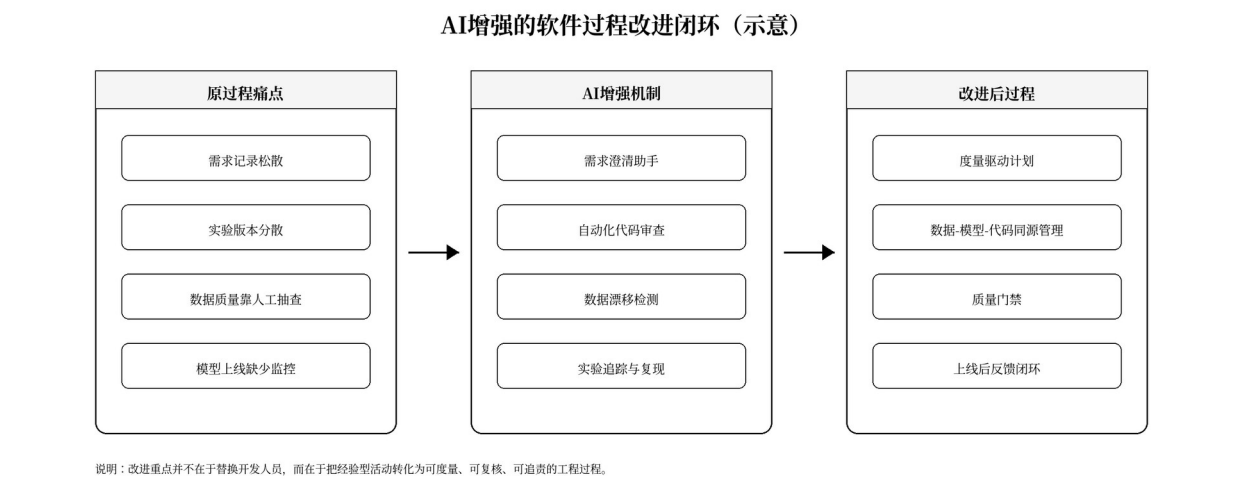


图 1 AI 增强的软件过程改进闭环

改进后过程由六个环节组成：需求建模、数据治理、模型开发、验证评估、发布交付、监控反馈。每个环节都设置输入、输出和质量门禁。需求建模输出任务说明与验收指标；数据治理输出数据版本和质量报告；模型开发输出代码、参数与训练日志；验证评估输出对比实验和误差分析；发布交付输出可视化看板或接口说明；监控反馈输出漂移报告和迭代建议。

过程环节	输入	输出	质量门禁
需求建模	课程要求、项目目标、用户场景	需求说明、指标定义、验收标准	指标必须覆盖总体误差与高流量 OD
数据治理	订单、天气、POI、日历	数据集版本、清洗日志、缺失率报告	缺失率超过 5% 必明理策略
模型开发	特征表、训练配置、基线模型	模型代码、参数、训练记录	代码、参数、数据版本必须绑定

验证评估	预测结果、真实值、场景 标签	对比表、图表、误差案例	至少包含 3 个基线与 1 组 消融实验
发布交付	评估通过的模型与报告	看板/API/演示材料	输出结果必须可解释、可 回溯
监控反馈	运行日志、反馈记录、漂 移指标	告警记录、迭代清单	性能下降超过 10% 触发 复核

表 2 改进后过程的输入、输出与质量门禁

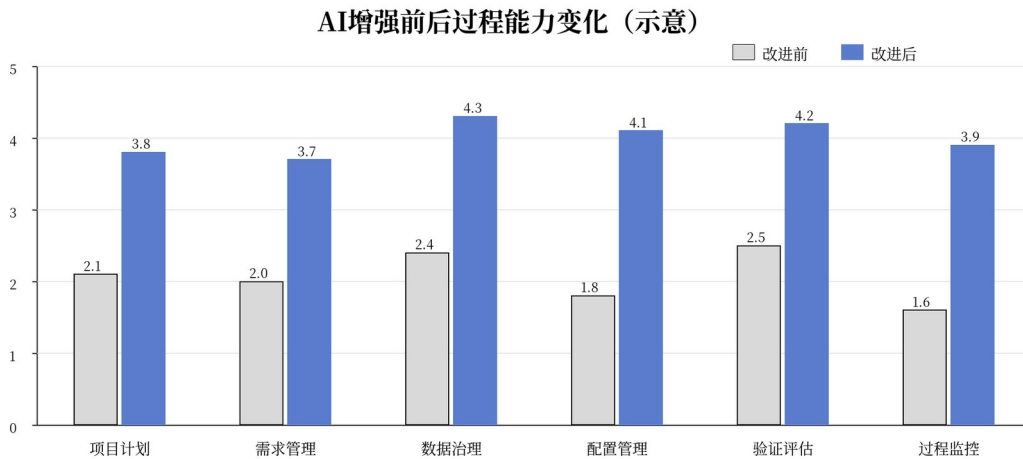
4 AI 增强机制设计

AI 增强不应被简单理解为“让大模型写代码”。在本方案中，AI 被嵌入四类高频活动。需求阶段，语言模型把自然语言需求改写为用户故事、验收指标和边界条件；开发阶段，代码审查助手检查数据泄露、硬编码路径、随机种子缺失等问题；评估阶段，日志分析助手汇总多轮实验差异；运维阶段，异常解释助手将漂移、误差尖峰和业务事件连接起来。

为了防止自动化偏见，所有 AI 生成内容都必须进入人工复核队列。需求文档不能直接由模型定稿，实验结论不能由模型单独确认，异常解释不能替代业务判断。AI 的价值在于压缩信息整理时间和暴露潜在缺陷，而不是把责任转移给工具。

5 过程能力提升评估

按照 0-5 的课程化能力刻度估算，改进前过程能力集中在 1.6-2.5 之间，说明团队能够完成基本开发，却缺少稳定复现和持续改进能力。改进后，数据治理、配置管理和验证评估提升最明显。尤其是数据治理从 2.4 提升到 4.3，原因在于版本记录、缺失率检查和样本切分规则被制度化。



注：评分采用 0-5 的课程化能力刻度，数值用于说明过程改进方向。

图 2 AI 增强前后过程能力变化

6 改进路线图

阶段	周期	关键任务	交付物	验收指标
阶段 A：过程基线固化	第 1 周	统一需求模板、数据目录和实验命名规则	过程基线文档、数据版本表	核心文档完整率达到 90%
阶段 B：自动化评估接入	第 2-3 周	编写评估脚本、生成固定评估脚本、实验报告模板、绑定模型配置	评估脚本、实验报告模板	复现实验耗时低于 20 分钟
阶段 C：AI 助治理	第 4 周	接入日志摘要、代码审查提示和异常解释草稿	AI 辅助审查清单、异常分析报告	人工复核后有效建议占比超过 60%

表 3 软件过程改进路线图

7 风险控制与局限性

改进方案也有边界。AI 辅助文档可能制造流畅但错误的解释，自动化脚本可能把错误规则稳定地重复执行，过程门禁也可能把团队拖入形式主义。一个项目若为了填写表格而牺牲验证质量，过程改进就会反噬工程效率。

因此，本方案把“轻量化”作为重要约束：每个新增过程项必须回答两个问题。它是否降低了复现成本？它是否提升了结果可信度？若答案都是否定的，该过程项就不应纳入课程项目。真正有效的软件过程不是文件越多越好，而是关键证据足够完整，决策路径足够清楚。

8 结论

本方案通过 AI 辅助需求、代码、实验和监控，把原本分散的软件活动改造为闭环过程。它不追求企业级平台的完整形态，而选择与课程项目规模相匹配的过程能力：可追踪、可、可、可改。智能交通 OD 预测这类 AI 软件而言，这种改进比单次模型精度提升更重要，因为模型会迭代，数据会漂移，场景会变化；只有过程被稳定下来，项目才不会在下一次实验中重新开始。

参考资料

- [1] ISO/IEC/IEEE 12207:2017, Systems and software engineering - Software life cycle processes.
- [2] ISO/IEC 33001, Information technology - Process assessment - Concepts and terminology.
- [3] NIST, Artificial Intelligence Risk Management Framework (AI RMF 1.0), 2023.
- [4] ISO/IEC 42001:2023, Artificial intelligence management system.